



Teleinformática e Redes 1

# **Camada de Enlace de Dados: Detecção e Correção de Erros**

Prof.º Geraldo P. Rocha Filho

geraldof@unb.br

Brasília  
2019



- O que é detecção e correção de erros?
- Código de detecção de erros
  - Bit de paridade
  - Checksum
  - CRC
- Código de correção de erros



**Relembrando...**

# Relembrando...





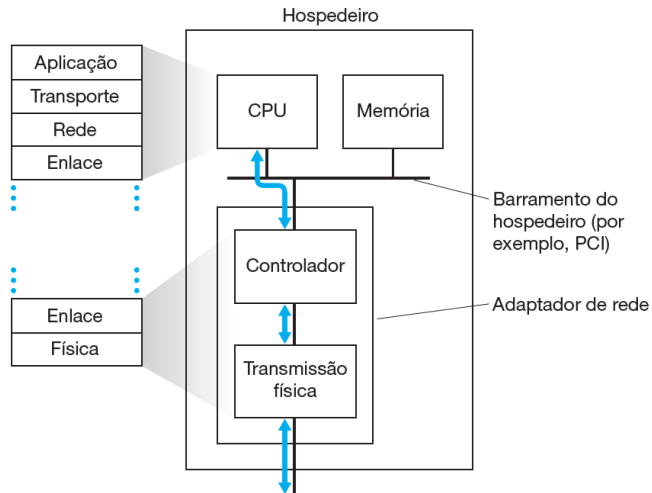
## **As principais funções da camada de enlace**

- Fornecer uma interface bem definida à camada superior
- Lidar com erros de transmissão
- Regular o fluxo de dados



## As principais funções da camada de enlace

- Fornecer uma interface bem definida à camada superior
- Lidar com erros de transmissão
- Regular o fluxo de dados

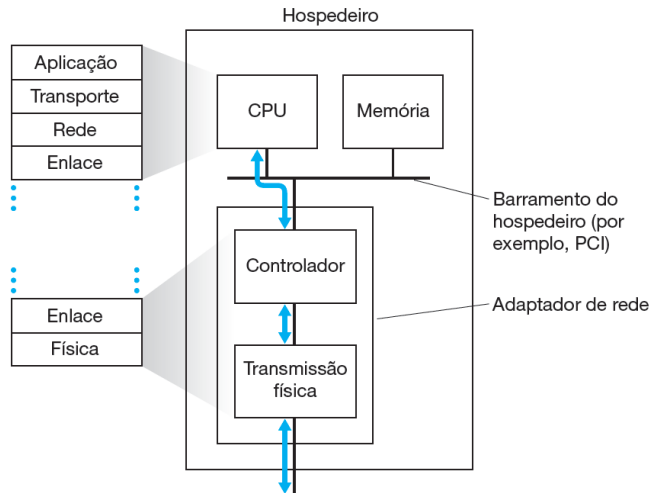




## As principais funções da camada de enlace

- Fornecer uma interface bem definida à camada superior
- Lidar com erros de transmissão
- Regular o fluxo de dados

Serviços sem conexão e sem/com confirmação  
e Serviço com conexão com confirmação

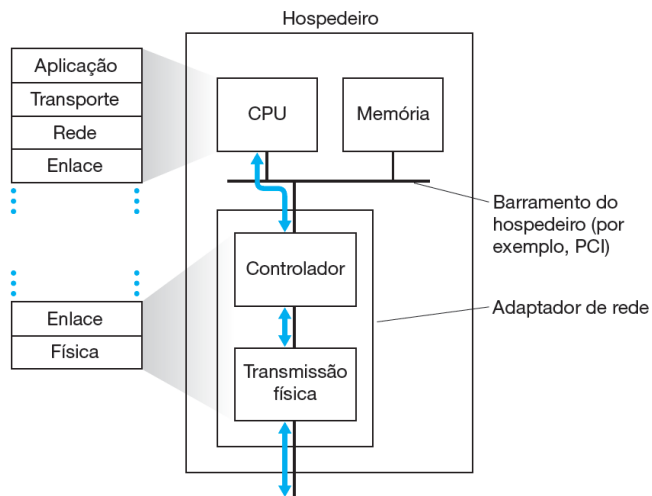




## As principais funções da camada de enlace

- Fornecer uma interface bem definida à camada superior
- Lidar com erros de transmissão
- Regular o fluxo de dados

Serviços sem conexão e sem/com confirmação  
e Serviço com conexão com confirmação



## Delimitação do quadro:

- Contagem de caracteres
- Bytes de flag com inserção de bytes
- Flags iniciais e finais, com inserção de bits
- Violações de codificação da camada física



# **Detecção e Correção de Erros**



- Erros de transmissão acontecem:
  - Na fibra óptica, **as taxas de erros são pequenas**, de modo que os erros de transmissão são uma ocorrência rara!



- Erros de transmissão acontecem:
  - Na fibra óptica, **as taxas de erros são pequenas**, de modo que os erros de transmissão são uma ocorrência rara!
  - Entretanto, outros canais, especialmente enlaces sem fios e antigos circuitos terminais, **as taxas de erros são maiores!**



- Erros de transmissão acontecem:
  - Na fibra óptica, **as taxas de erros são pequenas**, de modo que os erros de transmissão são uma ocorrência rara!
  - Entretanto, outros canais, especialmente enlaces sem fios e antigos circuitos terminais, **as taxas de erros são maiores!**
- **Conclusão:**

Erros vieram para ficar, então, como lidar com tais erros?



- Os projetistas de redes desenvolveram **duas estratégias** básicas para **lidar** com os erros:



- Os projetistas de redes desenvolveram **duas estratégias** básicas para **lidar** com os erros:
  - Inclusão de informação **redundante** para **corrigir** o erro



- Os projetistas de redes desenvolveram **duas estratégias** básicas para **lidar** com os erros:
  - Inclusão de informação **redundante** para **corrigir** o erro
  - Inclusão de informação **redundante** para **detectar** o erro
    - Receptor recebe apenas informações suficiente para comunicar que houve um erro!



- Os projetistas de redes desenvolveram **duas estratégias** básicas para **lidar** com os erros:
  - Inclusão de informação **redundante** para **corrigir** o erro
  - Inclusão de informação **redundante** para **detectar** o erro
    - Receptor recebe apenas informações suficiente para comunicar que houve um erro!
- Cada **estratégia** é adequada para um **determinado ambiente**





**Na fibra óptica, é mais vantajoso utilizar um código de detecção ou correção de erros?**

Detecção

Correção



**Na fibra óptica, é mais vantajoso utilizar um código de detecção ou correção de erros?**

Detecção

Correção

# Qual a melhor estratégia: Detecção ou Correção?



**Na fibra óptica, é mais vantajoso utilizar um código de detecção ou correção de erros?**

Detecção

Correção



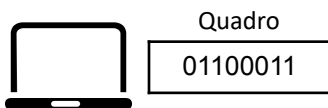
- Inclusão de informação redundante para **corrigir** o erro:
  - **Código de correção de erros**
    - Também chamado de **correção antecipada de erros**



- Inclusão de informação redundante para **corrigir** o erro:
  - **Código de correção de erros**
    - Também chamado de **correção antecipada de erros**
  - Usados em meios **menos** confiáveis, i.e., Enlace sem fio



- Inclusão de informação redundante para **corrigir** o erro:
  - **Código de correção de erros**
    - Também chamado de **correção antecipada de erros**
  - Usados em meios **menos** confiáveis, i.e., Enlace sem fio
  - Tenta descobrir qual era o bloco transmitido originalmente





- Inclusão de informação redundante para **corrigir** o erro:
  - **Código de correção de erros**
    - Também chamado de **correção antecipada de erros**
  - Usados em meios **menos** confiáveis, i.e., Enlace sem fio
  - Tenta descobrir qual era o bloco transmitido originalmente

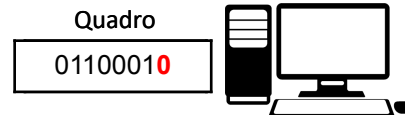


Quadro  
01100011





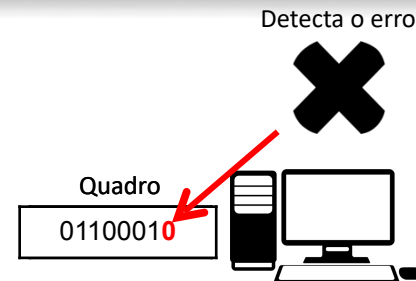
- Inclusão de informação redundante para **corrigir** o erro:
  - **Código de correção de erros**
    - Também chamado de **correção antecipada de erros**
  - Usados em meios **menos** confiáveis, i.e., Enlace sem fio
  - Tenta descobrir qual era o bloco transmitido originalmente







- Inclusão de informação redundante para **corrigir** o erro:
  - **Código de correção de erros**
    - Também chamado de **correção antecipada de erros**
  - Usados em meios **menos** confiáveis, i.e., Enlace sem fio
  - Tenta descobrir qual era o bloco transmitido originalmente





- Inclusão de informação redundante para **corrigir** o erro:
  - **Código de correção de erros**
    - Também chamado de **correção antecipada de erros**
  - Usados em meios **menos** confiáveis, i.e., Enlace sem fio
  - Tenta descobrir qual era o bloco transmitido originalmente



Quadro  
01100011





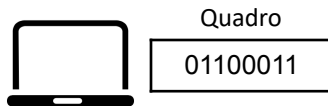
- Inclusão de informação redundante para **detectar** o erro:
  - Também conhecido como **Códigos de deteção de erros**



- Inclusão de informação redundante para **detectar** o erro:
  - Também conhecido como **Códigos de deteção de erros**
  - Usados em meios **confiáveis**, i.e. Fibra óptica

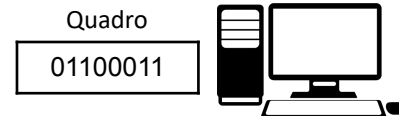


- Inclusão de informação redundante para **detectar** o erro:
  - Também conhecido como **Códigos de deteção de erros**
  - Usados em meios **confiáveis**, i.e. Fibra óptica
  - O bloco defeituoso é **retransmitido**



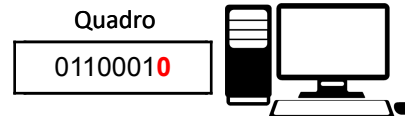


- Inclusão de informação redundante para **detectar** o erro:
  - Também conhecido como **Códigos de deteção de erros**
  - Usados em meios **confiáveis**, i.e. Fibra óptica
  - O bloco defeituoso é **retransmitido**



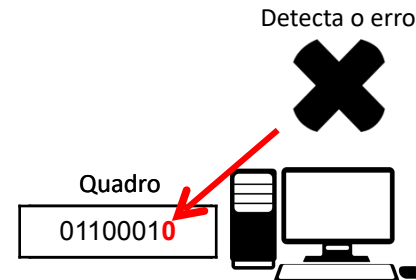


- Inclusão de informação redundante para **detectar** o erro:
  - Também conhecido como **Códigos de deteção de erros**
  - Usados em meios **confiáveis**, i.e. Fibra óptica
  - O bloco defeituoso é **retransmitido**





- Inclusão de informação redundante para **detectar** o erro:
  - Também conhecido como **Códigos de deteção de erros**
  - Usados em meios **confiáveis**, i.e. Fibra óptica
  - O bloco defeituoso é **retransmitido**







- Inclusão de informação redundante para **detectar** o erro:
  - Também conhecido como **Códigos de deteção de erros**
  - Usados em meios **confiáveis**, i.e. Fibra óptica
  - O bloco defeituoso é **retransmitido**



Retransmitir

Quadro

01100011





**Os códigos de correção e deteção de erro podem lidar com todos os erros possíveis?**



**Os códigos de correção e detecção de erro podem lidar com todos os erros possíveis?**

**Não, pois os **bits redundantes** que oferecem proteção provavelmente serão recebidos com erro, como os **bits de dados** (o que pode comprometer sua proteção)**



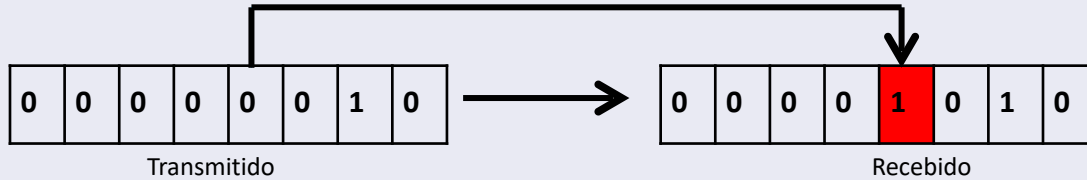
- Erros simples (um bit isolado)

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|

Transmitido



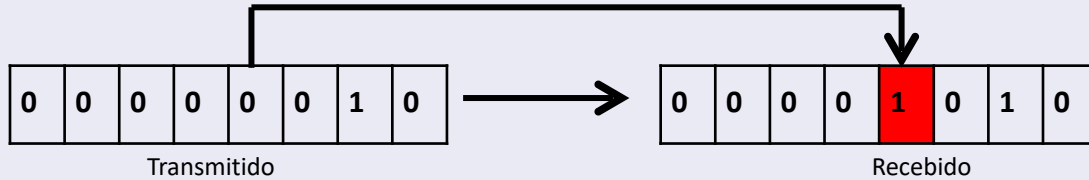
- Erros simples (um bit isolado)



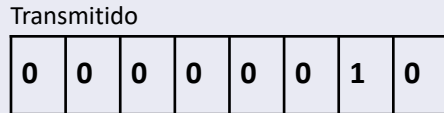
# Detecção e Correção de Erros – Tipos de erros:



- Erros simples (um bit isolado)



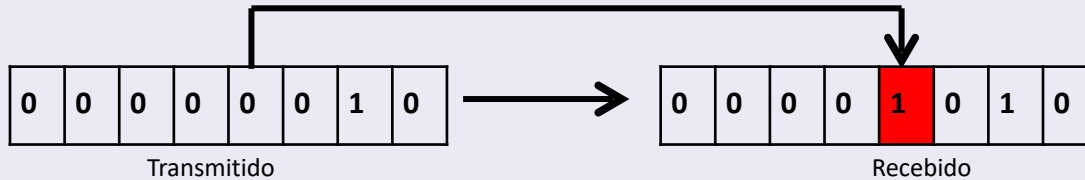
- Rajada de erros (sequência de bits corrompidos)



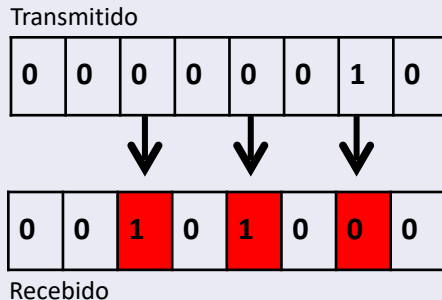
# Detecção e Correção de Erros – Tipos de erros:



- Erros simples (um bit isolado)



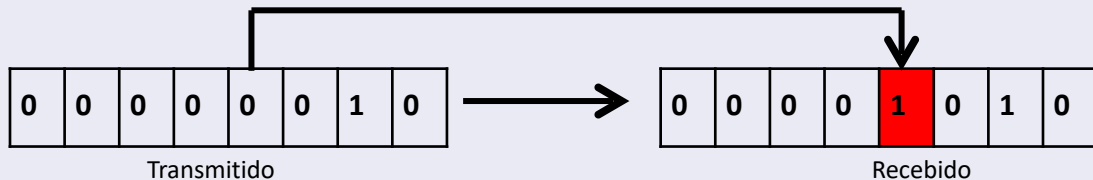
- Rajada de erros (sequência de bits corrompidos)



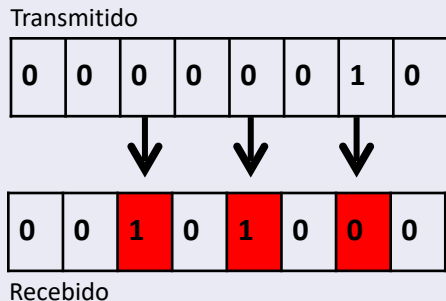
# Detecção e Correção de Erros – Tipos de erros:



- Erros simples (um bit isolado)



- Rajada de erros (sequência de bits corrompidos)



Outro erro?



## Rajada de erros *versus* erros simples!



**Os dados sempre são enviados em blocos de bits. Suponha que o tamanho do bloco seja 1.000 bits e que a taxa de erros seja 0,001 por bit. Qual tipo de erro é mais vantajoso?**



Os dados sempre são enviados em blocos de bits. Suponha que o tamanho do bloco seja 1.000 bits e que a taxa de erros seja 0,001 por bit. Qual tipo de erro é mais vantajoso?

- Se os erros **fossem independentes**, a maioria dos blocos teria um erro



**Os dados sempre são enviados em blocos de bits. Suponha que o tamanho do bloco seja 1.000 bits e que a taxa de erros seja 0,001 por bit. Qual tipo de erro é mais vantajoso?**

- Se os erros **fossem independentes**, a maioria dos blocos teria um erro
- Porém, se os **erros surgirem em rajadas de 100**, apenas um ou dois blocos em 100 será(ão) afetado(s), em média

# Rajada de erros *versus* erros simples!



Os dados sempre são enviados em blocos de bits. Suponha que o tamanho do bloco seja 1.000 bits e que a taxa de erros seja 0,001 por bit. Qual tipo de erro é mais vantajoso?

- Se os erros **fossem independentes**, a maioria dos blocos teria um erro
- Porém, se os **erros surgirem em rajadas de 100**, apenas um ou dois blocos em 100 será(ão) afetado(s), em média

Há uma maior probabilidade de ocorrer um erro se os erros **fossem independentes** ao invés de erros que **surgissem em rajadas**.

# Rajada de erros *versus* erros simples!



Os dados sempre são enviados em blocos de bits. Suponha que o tamanho do bloco seja 1.000 bits e que a taxa de erros seja 0,001 por bit. Qual tipo de erro é mais vantajoso?

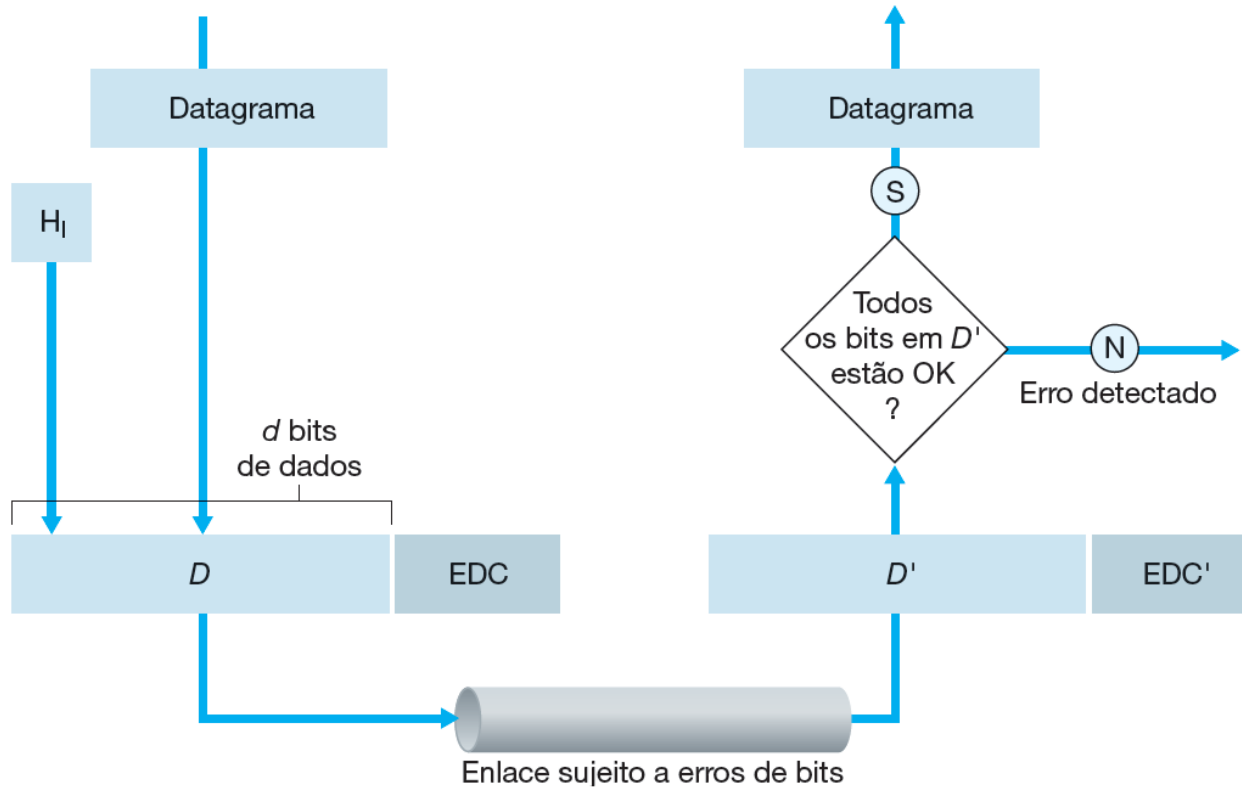
- Se os erros **fossem independentes**, a maioria dos blocos teria um erro
- Porém, se os **erros surgirem em rajadas de 100**, apenas um ou dois blocos em 100 será(ão) afetado(s), em média

Há uma maior probabilidade de ocorrer um erro se os erros **fossem independentes** ao invés de erros que **surgissem em rajadas**.

- Erros em rajada são mais difíceis de corrigir que os erros isolados

# **Códigos de Detecção de Erros!**

# Cenário de Detecção de Erros!





- **Utilizados** em enlaces em que a **taxa de erros** é **baixa**, i.e., fibra óptica e fio de cobre
- Três principais **códigos de detecção** de erros:
  - Bit de paridade
  - Checksums
  - Verificações de redundância cíclica (CRCs)





- Estratégia mais **simples** que permite a detecção de erros **individuais**



- Estratégia mais **simples** que permite a detecção de erros **individuais**
- Consiste em **acrescentar** aos dados um **único bit** de paridade!



- Estratégia mais **simples** que permite a detecção de erros **individuais**
- Consiste em **acrescentar** aos dados um **único bit** de paridade!
- O bit de paridade é **escolhido** de forma que:
  - O número de bits 1 transmitido seja par;
  - OU, este número seja ímpar



- Estratégia mais **simples** que permite a detecção de erros **individuais**
- Consiste em **acrescentar** aos dados um **único bit** de paridade!
- O bit de paridade é **escolhido** de forma que:
  - O número de bits 1 transmitido seja par;
  - OU, este número seja ímpar
- **Implementado** em nível de *hardware* ou *software*?



- Estratégia mais **simples** que permite a detecção de erros **individuais**
- Consiste em **acrescentar** aos dados um **único bit** de paridade!
- O bit de paridade é **escolhido** de forma que:
  - O número de bits 1 transmitido seja par;
  - OU, este número seja ímpar
- **Implementado** em nível de *hardware* ou *software*?
- Exemplo:
  - Paridade par: 0101101 => 0101101**0**
  - Paridade ímpar: 0101101 => 0101101**1**

# Bit de paridade – Funcionamento



Transmissor

01100011



Receptor



# Bit de paridade – Funcionamento



Transmissor



Receptor



|          |   |
|----------|---|
| 01100011 | 0 |
|----------|---|

# Bit de paridade – Funcionamento



Transmissor



Receptor



01100011

Retira o bit de  
paridade

0



# Bit de paridade – Funcionamento



Transmissor



Receptor



01100011

Retira o bit de  
paridade

0

Recalcula o bit  
de paridade

0

# Bit de paridade – Funcionamento



Transmissor



Receptor

01100011



Retira o bit de  
paridade

0

OK!

Recalcula o bit  
de paridade

0

# Bit de paridade – Funcionamento com erro



Transmissor

01100011



Receptor



# Bit de paridade – Funcionamento com erro



Transmissor



Receptor



|          |   |
|----------|---|
| 01100001 | 0 |
|----------|---|

# Bit de paridade – Funcionamento com erro



Transmissor



Receptor



01100001

Retira o bit de  
paridade

0

# Bit de paridade – Funcionamento com erro



Transmissor



Receptor



01100001

Retira o bit de  
paridade

0

Recalcula o bit  
de paridade

1

# Bit de paridade – Funcionamento com erro



Transmissor



Retransmite

Receptor



# Bit de paridade – Funcionamento com erro



Transmissor



Retransmite

Receptor



**Cenário em que o bit de paridade não funciona?**



# Bit de paridade – Funcionamento com erro



Transmissor



Retransmite

Receptor



## Cenário em que o bit de paridade não funciona?

- Bit de paridade vier alterado?
- Se houver mais de dois bits alterados?



**Com o bit de paridade é possível saber qual o bit que possui erro?**

Sim

Não



**Com o bit de paridade é possível saber qual o bit que possui erro?**

Sim

Não



- Limitação do bit de paridade?



- Limitação do bit de paridade?

Detecta apenas erros de um único bit!



- Limitação do bit de paridade?

Detecta apenas erros de um único bit!

- E para resolver erros em rajadas?



- Limitação do bit de paridade?

Detecta apenas erros de um único bit!

- E para resolver erros em rajadas?
  - Entrelaçamento



- Entrelaçamento:
  - Os dados são formatados na forma de matrizes
  - Calcula-se os bits de paridade sobre os dados em uma ordem diferente
    - Neste caso, os bits de paridade são calculados para cada coluna
  - Envia todos os bits de dados, enviando **as linhas de cima para baixo** e os bits em cada linha da **esquerda para a direita**
  - Na última linha, envia-se os  $n$  bits de paridade



# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

# Entrelaçamento – Erros em rajadas!



Transmissor

7

1 0 0 1 1 1 0  
1 1 0 0 1 0 1  
1 1 1 0 1 0 0  
1 1 1 0 1 1 1  
1 1 0 1 1 1 1  
1 1 1 0 0 1 0  
1 1 0 1 0 1 1

# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

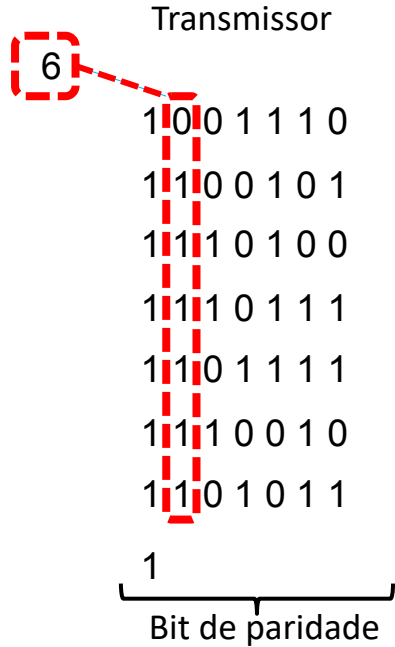
1 1 1 0 0 1 0

1 1 0 1 0 1 1

1

└──────────┘  
Bit de paridade

# Entrelaçamento – Erros em rajadas!



# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

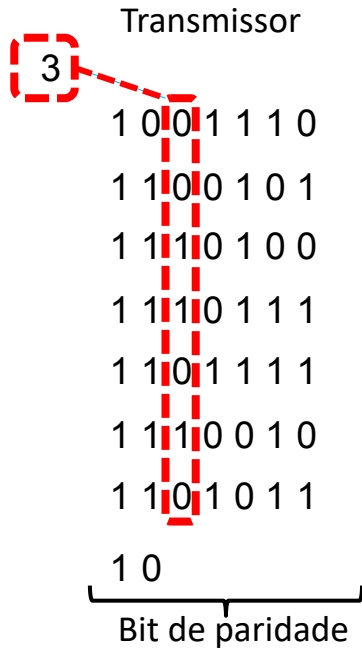
1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0

└──────────┘  
Bit de paridade

# Entrelaçamento – Erros em rajadas!



# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

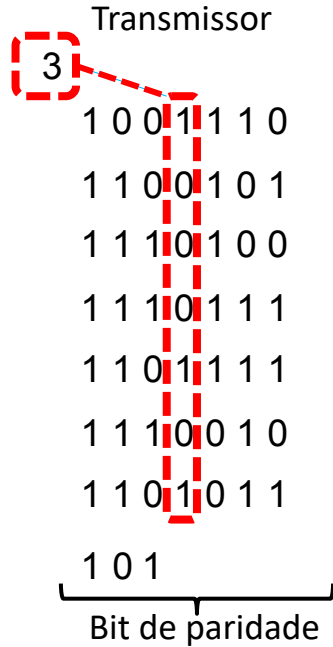
1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1

└──────────┘  
Bit de paridade

# Entrelaçamento – Erros em rajadas!





# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

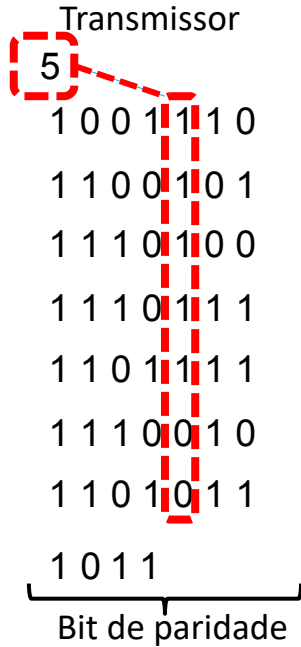
1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1

└──────────┘  
Bit de paridade

# Entrelaçamento – Erros em rajadas!



# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

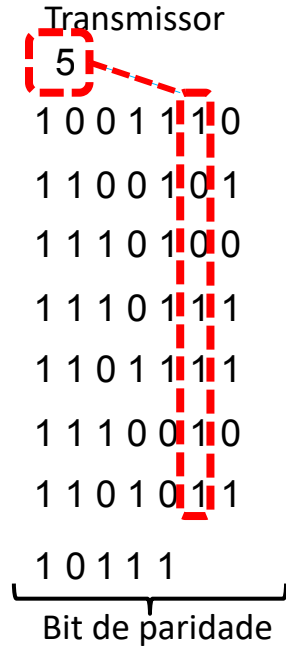
1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1

Bit de paridade

# Entrelaçamento – Erros em rajadas!



# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

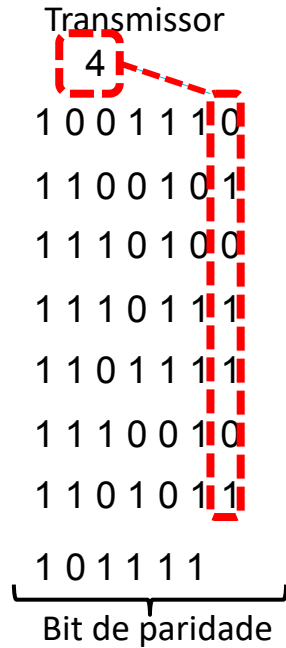
1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1

Bit de paridade

# Entrelaçamento – Erros em rajadas!



# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1 0

Bit de paridade

# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1 0

Bit de paridade

Transmissão  
Canal →

Receptor

1 0 0 1 1 1 0

1 1 0 0 0 1 1

1 1 0 1 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1 0

Erro em  
rajada



# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1 0

Bit de paridade

Transmissão  
Canal →

Receptor

1 0 0 1 1 1 0

1 1 0 0 0 1 1

1 1 0 1 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

Erro em  
rajada

Retira o bit de  
paridade

1 0 1 1 1 1 0

# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1 0

Bit de paridade

Transmissão  
Canal →

Receptor

1 0 0 1 1 1 0

1 1 0 0 0 1 1

1 1 0 1 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 0 0 0 0 0

Erro em  
rajada

Retira o bit de  
paridade

1 0 1 1 1 1 0

# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1 0

Bit de paridade

Transmissão  
Canal →

Receptor

1 0 0 1 1 1 0

1 1 0 0 0 1 1

1 1 0 1 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

Erro em  
rajada

Retira o bit de  
paridade

1 0 1 1 1 1 0

Recalcula o bit  
de paridade

1 0 0 0 0 0 0

# Entrelaçamento – Erros em rajadas!



Transmissor

1 0 0 1 1 1 0

1 1 0 0 1 0 1

1 1 1 0 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

1 0 1 1 1 1 0

Bit de paridade

Transmissão  
Canal →

Receptor

1 0 0 1 1 1 0

1 1 0 0 0 1 1

1 1 0 1 1 0 0

1 1 1 0 1 1 1

1 1 0 1 1 1 1

1 1 1 0 0 1 0

1 1 0 1 0 1 1

Erro em  
rajada

Retira o bit de  
paridade

1 0 1 1 1 1 0

Erro de quantos  
bits?

Recalcula o bit  
de paridade

1 0 0 0 0 0 0



- O termo checksum é usado para indicar um **grupo** de bits de **verificação** associados a uma mensagem
  - Um grupo de bits de paridade é um exemplo de checksum, i.e., entrelaçamento



- O termo checksum é usado para indicar um **grupo** de bits de **verificação** associados a uma mensagem
  - Um grupo de bits de paridade é um exemplo de checksum, i.e., entrelaçamento
- O checksum é adicionado no **final** da mensagem, como complemento da **função de soma**



- O termo checksum é usado para indicar um **grupo** de bits de **verificação** associados a uma mensagem
  - Um grupo de bits de paridade é um exemplo de checksum, i.e., entrelaçamento
- O checksum é adicionado no **final** da mensagem, como complemento da **função de soma**
- Com isso, os erros podem ser detectados somando a palavra inteira de código recebida, tanto bits de dados quanto a soma de verificação
  - Se o **resultado** for **zero**, nenhum **erro** foi **detectado**



- O termo checksum é usado para indicar um **grupo** de bits de **verificação** associados a uma mensagem
  - Um grupo de bits de paridade é um exemplo de checksum, i.e., entrelaçamento
- O checksum é adicionado no **final** da mensagem, como complemento da **função de soma**
- Com isso, os erros podem ser detectados somando a palavra inteira de código recebida, tanto bits de dados quanto a soma de verificação
  - Se o **resultado** for **zero**, nenhum **erro** foi **detectado**
- Exemplo: Protocolo IP – soma de verificação de 16 bits





## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- **Envia** a mensagem junto com o checksum

**Mensagem a ser transmitida:**

1 0 1 0 1 0 0 1 0 0 1 1 1 0 0 1



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- **Envia** a mensagem junto com o checksum

Mensagem a ser transmitida:

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1 \\ 0\ 0\ 1\ 1\ 1\ 0\ 0\ 1 \end{array} \begin{array}{l} \\ + \end{array}$$

---



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- **Envia** a mensagem junto com o checksum

Mensagem a ser transmitida:

$$\begin{array}{r} 10101001 \\ 00111001^+ \\ \hline 11100010 \end{array}$$



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- **Envia** a mensagem junto com o checksum

Mensagem a ser transmitida:

1 0 1 0 1 0 0 1

0 0 1 1 1 0 0 1<sup>+</sup>

---

1 1 1 0 0 0 1 0

0 0 0 1 1 1 0 1

Complemento da soma - Checksum



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- **Envia** a mensagem junto com o checksum

Mensagem a ser transmitida:

1 0 1 0 1 0 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1



Mensagem codificada para ser enviada



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- Se o **checksum** for igual a **zero**, dados aceitos

## Mensagem Recebida:

1 0 1 0 1 0 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1

└────────────────────────────────┘  
Mensagem decodificada



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- Se o **checksum** for igual a **zero**, dados aceitos

Mensagem Recebida:

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1 \\ 0\ 0\ 1\ 1\ 1\ 0\ 0\ 1 \\ \hline 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1 \end{array}$$

Mensagem decodificada



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- Se o **checksum** for igual a **zero**, dados aceitos

## Mensagem Recebida:

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1 \\ 0\ 0\ 1\ 1\ 1\ 0\ 0\ 1 \\ \hline 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1 \\ \hline 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1 \end{array}$$

└──────────────────┘  
Mensagem decodificada





## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- Se o **checksum** for igual a **zero**, dados aceitos

## Mensagem Recebida:

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1\ 0\ 0\ 1 \\ 0\ 0\ 1\ 1\ 1\ 0\ 0\ 1 \\ \hline 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1 \\ \hline 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1 \\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0 \end{array}$$

Checksum zero, sem erro!

Mensagem decodificada



## Passo a passo:

- **Divide** a mensagem em  $k$  segmentos de  $n$  bits;
- **Soma** os  $k$  segmentos
- **Forma** o checksums com o **complemento da soma**
- Se o **checksum** for igual a **zero**, dados aceitos

## Mensagem Recebida:

1 0 1 0 1 0 0 1 0 0 1 1 1 0 0 1

Mensagem decodificada



**Erros que passaram pelos bits de verificação podem ser detectados?**

Sim

Não



**Erros que passaram pelos bits de verificação podem ser detectados?**

Sim

Não



**Erros que passaram pelos bits de verificação podem ser detectados?**

Sim

Não

- Há algum caso que o checksum não detecta erros?



**Erros que passaram pelos bits de verificação podem ser detectados?**

Sim

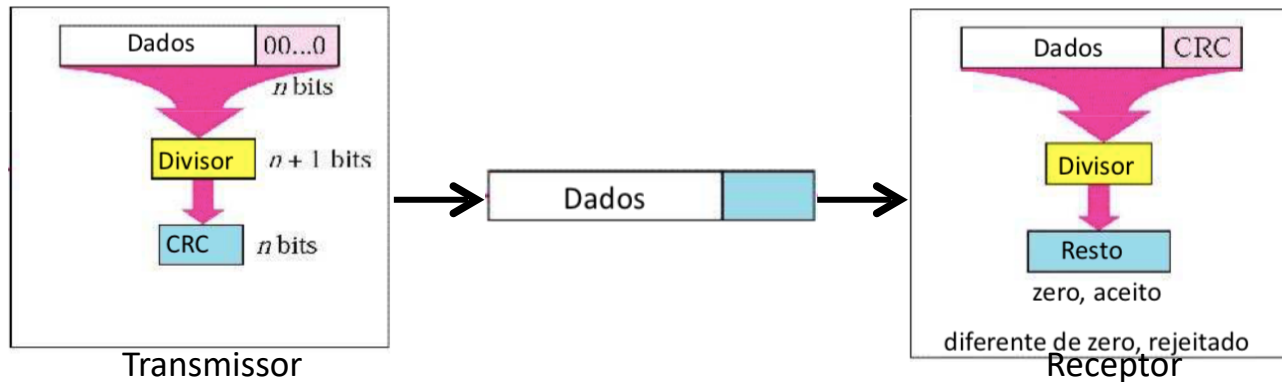
Não

- Há algum caso que o checksum não detecta erros?

Não detecta a exclusão ou o acréscimo de dados zero, e oferece pouca proteção contra pedaços da mensagem em que partes de dois pacotes são reunidas



- Embora os dois esquemas anteriores possam ser adequados, na prática há uma terceira **estratégia** mais **robusta** para detectar erros
- Tal estratégia é o **Código de Redundância Cíclica**: CRC (*Cyclic Redundancy Check*),
  - O CRC também conhecido como código polinomial!





- Ideia:
  - Quando o CRC é empregado, o **transmissor** e o **receptor** devem concordar em relação a um **polinômio gerador**,  $G(x)$





- Ideia:
  - Quando o CRC é empregado, o **transmissor** e o **receptor** devem concordar em relação a um **polinômio gerador**,  $G(x)$
  - Tanto o bit de mais **alta** ordem quanto o de mais **baixa** ordem do  $G(x)$  devem ser **iguais a 1**



- Ideia:
  - Quando o CRC é empregado, o **transmissor** e o **receptor** devem concordar em relação a um **polinômio gerador**,  $G(x)$
  - Tanto o bit de mais **alta** ordem quanto o de mais **baixa** ordem do  $G(x)$  devem ser **iguais a 1**
  - Para **calcular** o CRC de um quadro com  $m$  bits, o quadro deve ter **mais bits** do que o  $G(x)$ . Isto é, bits do quadro  $\geq G(x)$ . **Por quê?**



- Ideia:
  - Quando o CRC é empregado, o **transmissor** e o **receptor** devem concordar em relação a um **polinômio gerador**,  $G(x)$
  - Tanto o bit de mais **alta** ordem quanto o de mais **baixa** ordem do  $G(x)$  devem ser **iguais a 1**
  - Para **calcular** o CRC de um quadro com  $m$  bits, o quadro deve ter **mais bits** do que o  $G(x)$ . Isto é, bits do quadro  $\geq G(x)$ . **Por quê?**
  - **Adicionar** o CRC gerado no final do quadro e **enviar**



- Ideia:
  - Quando o CRC é empregado, o **transmissor** e o **receptor** devem concordar em relação a um **polinômio gerador**,  $G(x)$
  - Tanto o bit de mais **alta** ordem quanto o de mais **baixa** ordem do  $G(x)$  devem ser **iguais a 1**
  - Para **calcular** o CRC de um quadro com  $m$  bits, o quadro deve ter **mais bits** do que o  $G(x)$ . Isto é, bits do quadro  $\geq G(x)$ . **Por quê?**
  - **Adicionar** o CRC gerado no final do quadro e **enviar**
  - Quando obtiver o quadro verificado, o receptor tentará dividi-lo por  $G(x)$



- Ideia:
  - Quando o CRC é empregado, o **transmissor** e o **receptor** devem concordar em relação a um **polinômio gerador**,  $G(x)$
  - Tanto o bit de mais **alta** ordem quanto o de mais **baixa** ordem do  $G(x)$  devem ser **iguais a 1**
  - Para **calcular** o CRC de um quadro com  $m$  bits, o quadro deve ter **mais bits** do que o  $G(x)$ . Isto é, bits do quadro  $\geq G(x)$ . **Por quê?**
  - **Adicionar** o CRC gerado no final do quadro e **enviar**
  - Quando obtiver o quadro verificado, o receptor tentará dividi-lo por  $G(x)$
  - A existência de resto indica que houve um erro de transmissão

# Código de Redundância Cíclica - Transmissor



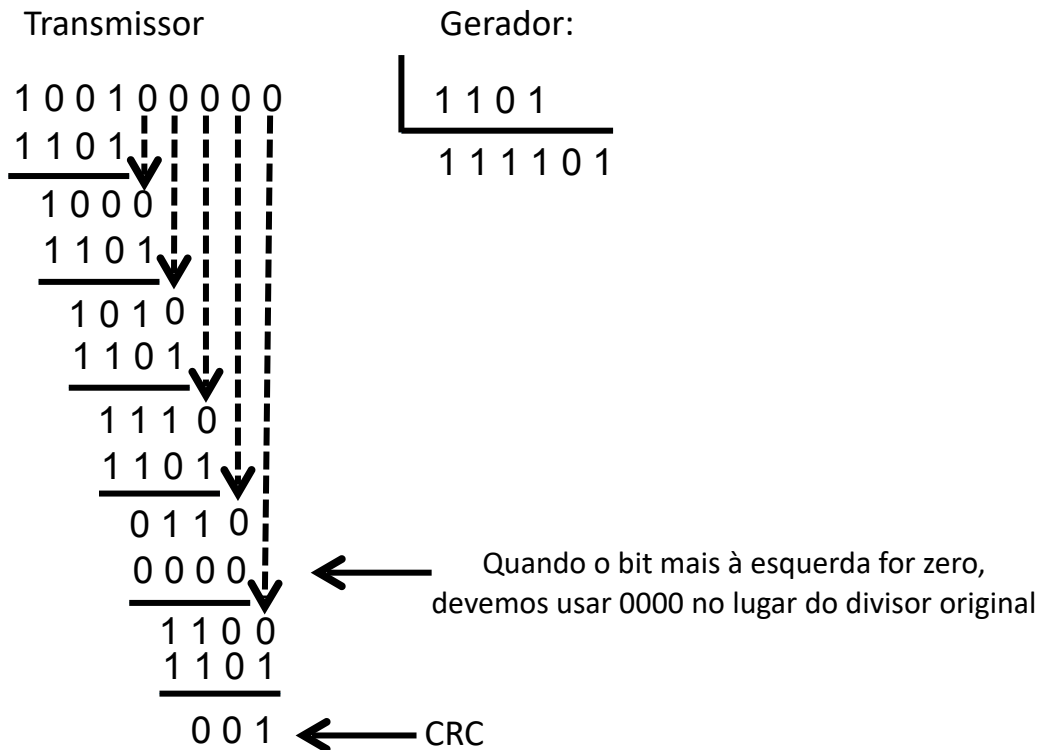
Transmissor

1 0 0 1 0 0 0 0 0

Gerador:

1 1 0 1

# Código de Redundância Cíclica - Transmissor



# Código de Redundância Cíclica - Receptor



Transmissor

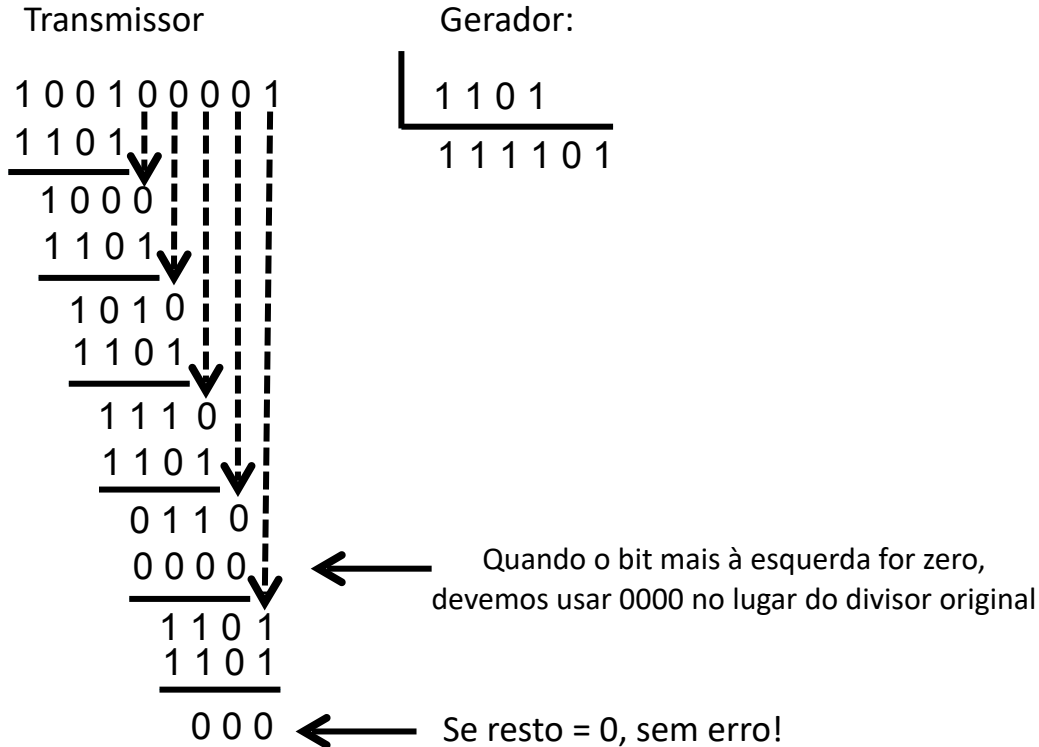
1 0 0 1 0 0 0 0 1

Gerador:

1 1 0 1



# Código de Redundância Cíclica - Receptor



# **Códigos de Correção de Erros!**



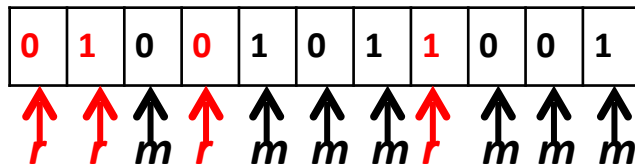
- No código de Hamming, vários *bits* de **paridade/verificação** são acrescentados usando regras especiais
  - Isto é, adiciona redundância



- No código de Hamming, vários *bits* de **paridade/verificação** são acrescentados usando regras especiais
  - Isto é, adiciona redundância
- Com esta **redundância** é possível corrigir erros

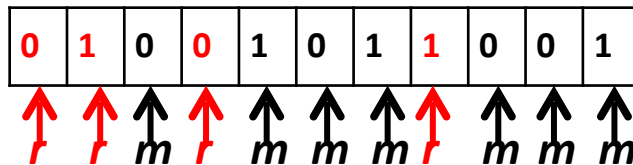


- No código de Hamming, vários *bits* de **paridade/verificação** são acrescentados usando regras especiais
  - Isto é, adiciona redundância
- Com esta **redundância** é possível corrigir erros
- Para isso, um quadro consiste de ***m*** bits de dados e ***r*** bits redundantes





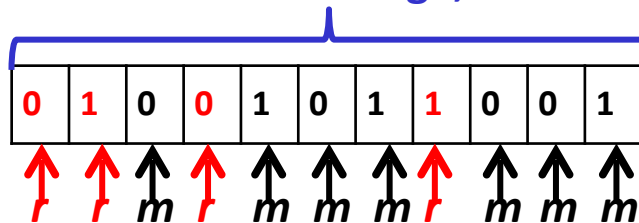
- No código de Hamming, vários *bits* de **paridade/verificação** são acrescentados usando regras especiais
  - Isto é, adiciona redundância
- Com esta **redundância** é possível corrigir erros
- Para isso, um quadro consiste de ***m*** bits de dados e ***r*** bits redundantes
- O tamanho total ***n*** é dado por ***m+r***;





- No código de Hamming, vários *bits* de **paridade/verificação** são acrescentados usando regras especiais
  - Isto é, adiciona redundância
- Com esta **redundância** é possível corrigir erros
- Para isso, um quadro consiste de  **$m$**  bits de dados e  **$r$**  bits redundantes
- O tamanho total  **$n$**  é dado por  **$m+r$** ;
- A unidade de  **$n$**  bits é chamada **palavra de código**

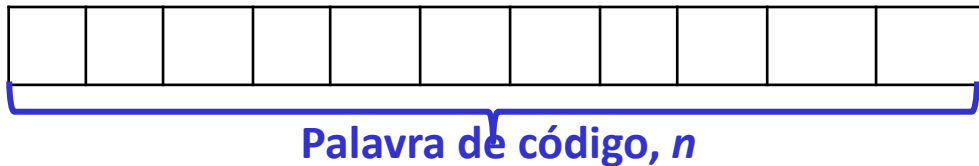
Palavra de código,  $n$





## Ideia:

- Os bits da **palavra de código** são numerados
  - Começando com o bit 1 da extremidade esquerda, o bit 2 imediatamente à sua direita e assim por diante

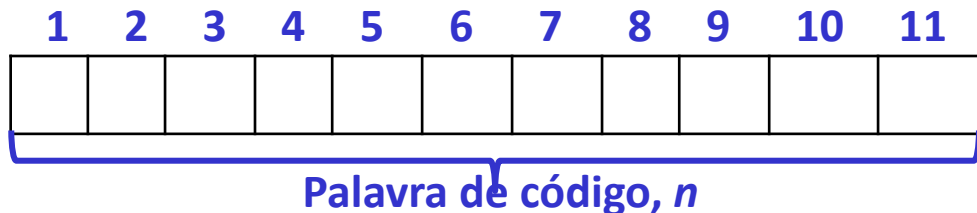






## Ideia:

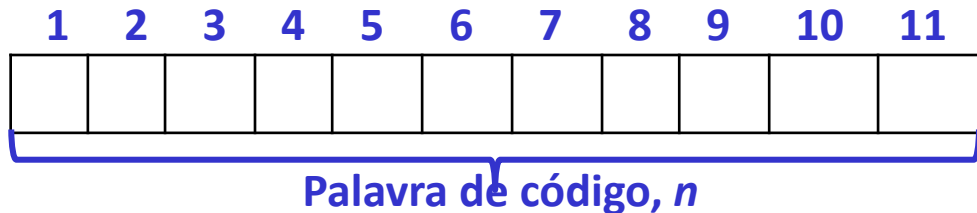
- Os bits da **palavra de código** são numerados
  - Começando com o bit 1 da extremidade esquerda, o bit 2 imediatamente à sua direita e assim por diante





## Ideia:

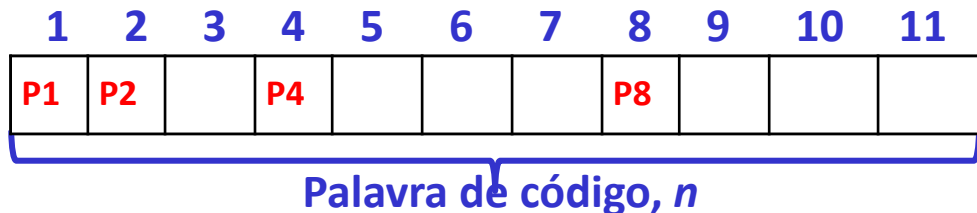
- Os bits da **palavra de código** são numerados
  - Começando com o bit 1 da extremidade esquerda, o bit 2 imediatamente à sua direita e assim por diante
- Bits que são potência de dois são de **verificação**





## Ideia:

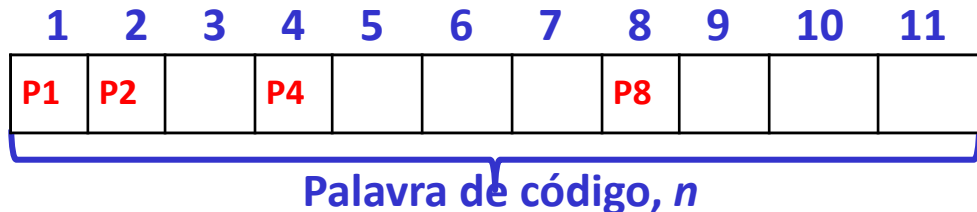
- Os bits da **palavra de código** são numerados
  - Começando com o bit 1 da extremidade esquerda, o bit 2 imediatamente à sua direita e assim por diante
- Bits que são potência de dois são de **verificação**





## Ideia:

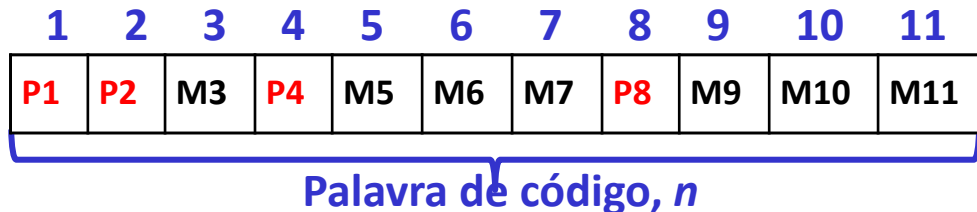
- Os bits da **palavra de código** são numerados
  - Começando com o bit 1 da extremidade esquerda, o bit 2 imediatamente à sua direita e assim por diante
- Bits que são potência de dois são de **verificação**
- Os demais são bits de **dados**





## Ideia:

- Os bits da **palavra de código** são numerados
  - Começando com o bit 1 da extremidade esquerda, o bit 2 imediatamente à sua direita e assim por diante
- Bits que são potência de dois são de **verificação**
- Os demais são bits de **dados**





## Ideia (cont.):

- Na palavra de código, um bit de **dados** (**M11**) pode ser incluído em vários bits de **verificação** (**P8, P2 e P1**) para realizar o código de hamming!
- Como isso pode ser feito?

|           |           |    |           |    |    |    |           |    |     |     |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|
| <b>P1</b> | <b>P2</b> | M3 | <b>P4</b> | M5 | M6 | M7 | <b>P8</b> | M9 | M10 | M11 |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|



## Ideia (cont.):

- Na palavra de código, um bit de **dados** (**M11**) pode ser incluído em vários bits de **verificação** (**P8, P2 e P1**) para realizar o código de hamming!
- Como isso pode ser feito?

Para saber para quais bits de **verificação** o bit na posição (k) contribui, reescreva o k como a soma das potências de 2.

|           |           |    |           |    |    |    |           |    |     |     |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|
| <b>P1</b> | <b>P2</b> | M3 | <b>P4</b> | M5 | M6 | M7 | <b>P8</b> | M9 | M10 | M11 |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|



## Ideia (cont.):

- Na palavra de código, um bit de **dados** (**M11**) pode ser incluído em vários bits de **verificação** (**P8, P2 e P1**) para realizar o código de hamming!
- Como isso pode ser feito?

Para saber para quais bits de **verificação** o bit na posição (k) contribui, reescreva o k como a soma das potências de 2.

– Para  $k = 5$ ?

|           |           |    |           |    |    |    |           |    |     |     |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|
| <b>P1</b> | <b>P2</b> | M3 | <b>P4</b> | M5 | M6 | M7 | <b>P8</b> | M9 | M10 | M11 |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|





## Ideia (cont.):

- Na palavra de código, um bit de **dados** (**M11**) pode ser incluído em vários bits de **verificação** (**P8**, **P2** e **P1**) para realizar o código de hamming!
- Como isso pode ser feito?

Para saber para quais bits de **verificação** o bit na posição (k) contribui, reescreva o k como a soma das potências de 2.

– Para  $k = 5$ ?  $5 = 4 + 1$

|           |           |    |           |    |    |    |           |    |     |     |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|
| <b>P1</b> | <b>P2</b> | M3 | <b>P4</b> | M5 | M6 | M7 | <b>P8</b> | M9 | M10 | M11 |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|



## Ideia (cont.):

- Na palavra de código, um bit de **dados** (**M11**) pode ser incluído em vários bits de **verificação** (**P8, P2 e P1**) para realizar o código de hamming!
- Como isso pode ser feito?

Para saber para quais bits de **verificação** o bit na posição (k) contribui, reescreva o k como a soma das potências de 2.

- Para  $k = 5$ ?  $5 = 4 + 1$
- Para  $k = 10$ ?

|           |           |    |           |    |    |    |           |    |     |     |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|
| <b>P1</b> | <b>P2</b> | M3 | <b>P4</b> | M5 | M6 | M7 | <b>P8</b> | M9 | M10 | M11 |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|



## Ideia (cont.):

- Na palavra de código, um bit de **dados** (**M11**) pode ser incluído em vários bits de **verificação** (**P8**, **P2** e **P1**) para realizar o código de hamming!
- Como isso pode ser feito?

Para saber para quais bits de **verificação** o bit na posição (k) contribui, reescreva o k como a soma das potências de 2.

- Para  $k = 5$ ?  $5 = 4 + 1$
- Para  $k = 10$ ?  $10 = 8 + 2$

|           |           |    |           |    |    |    |           |    |     |     |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|
| <b>P1</b> | <b>P2</b> | M3 | <b>P4</b> | M5 | M6 | M7 | <b>P8</b> | M9 | M10 | M11 |
|-----------|-----------|----|-----------|----|----|----|-----------|----|-----|-----|



## Ideia (cont.):

- O código de Hamming é definido pelo número de bits utilizados na forma  $\text{hamming}(n, m)$



## Ideia (cont.):

O que é?

- O código de Hamming é definido pelo número de bits utilizados na forma  $\text{hamming}(n, m)$



## Ideia (cont.):

- O código de Hamming é definido pelo número de bits utilizados na forma  $\text{hamming}(n, m)$ .

[O que é?]

[O que é?]



## Ideia (cont.):

- O código de Hamming é definido pelo número de bits utilizados na forma  $\text{hamming}(n, m)$ .  
[O que é?] [O que é?]
- **Importante:** Cada bit de verificação força a paridade de um conjunto de bits



## Ideia (cont.):

- O código de Hamming é definido pelo número de bits utilizados na forma  $\text{hamming}(n, m)$ .
  - O que é?
  - O que é?
- **Importante:** Cada bit de verificação força a paridade de um conjunto de bits
- Exemplo de um código de hamming(11, 7):
  - Usa um total de 11 bits
  - 7 bits de dados
  - 4 bits de verificação





- Ex. do Código de hamming(11, 7)!
  - Primeiro: Encontrar o conjunto de bits de dados para cada bit de verificação. Por exemplo, **P1** = M3 + M5 + M7 + M9 + M11
  - Isso é feito pois cada bit de verificação força a paridade de um conjunto de bits (paridade par ou ímpar) -> detectar o erro e corrigir!

|    |    |    |    |    |    |    |    |    |     |     |
|----|----|----|----|----|----|----|----|----|-----|-----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |



- Ex. do Código de hamming(11, 7)!
  - Primeiro: Encontrar o conjunto de bits de dados para cada bit de verificação. Por exemplo, **P1** = M3 + M5 + M7 + M9 + M11
  - Isso é feito pois cada bit de verificação força a paridade de um conjunto de bits (paridade par ou ímpar) -> detectar o erro e corrigir!

|    |    |    |    |    |    |    |    |    |     |     |
|----|----|----|----|----|----|----|----|----|-----|-----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

- Como encontrar esse padrão?

$$\text{P1} = \text{M3} + \text{M5} + \text{M7} + \text{M9} + \text{M11}$$

$$\text{P2} = \text{M3} + \text{M6} + \text{M7} + \text{M10} + \text{M11}$$

$$\text{P4} = \text{M5} + \text{M6} + \text{M7}$$

$$\text{P8} = \text{M9} + \text{M10} + \text{M11}$$

# Código de Hamming – Exemplo 1



- Maneira correta!
  - Reescrever cada **bit de dado** como a soma das potência de 2

|    |    |    |    |    |    |    |    |    |     |     |
|----|----|----|----|----|----|----|----|----|-----|-----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |



- Maneira correta!
  - Reescrever cada **bit de dado** como a soma das potência de 2

$$M3 = P2 + P1$$

$$M5 = P4 + P1$$

$$M6 = P4 + P2$$

$$M7 = P4 + P2 + P1$$

$$M9 = P8 + P1$$

$$M10 = P8 + P2$$

$$M11 = P8 + P2 + P1$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Maneira correta!
  - Reescrever cada **bit de dado** como a soma das potência de 2

$$M3 = P2 + P1$$

$$M5 = P4 + P1$$

$$M6 = P4 + P2$$

$$M7 = P4 + P2 + P1$$

$$M9 = P8 + P1$$

$$M10 = P8 + P2$$

$$M11 = P8 + P2 + P1$$

Combinação  
→

$P1 = ?$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Maneira correta!

- Reescrever cada **bit de dado** como a soma das potência de 2

$$M3 = P2 + P1$$

$$M5 = P4 + P1$$

$$M6 = P4 + P2$$

$$M7 = P4 + P2 + P1$$

$$M9 = P8 + P1$$

$$M10 = P8 + P2$$

$$M11 = P8 + P2 + P1$$

Combinação  
→

$$P1 = M3 + M5 + M7 + M9 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Maneira correta!

- Reescrever cada **bit de dado** como a soma das potência de 2

$$M3 = P2 + P1$$

$$M5 = P4 + P1$$

$$M6 = P4 + P2$$

$$M7 = P4 + P2 + P1$$

$$M9 = P8 + P1$$

$$M10 = P8 + P2$$

$$M11 = P8 + P2 + P1$$

Combinação  
→

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = ?$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Maneira correta!
  - Reescrever cada **bit de dado** como a soma das potência de 2

$$M3 = P2 + P1$$

$$M5 = P4 + P1$$

$$M6 = P4 + P2$$

$$M7 = P4 + P2 + P1$$

$$M9 = P8 + P1$$

$$M10 = P8 + P2$$

$$M11 = P8 + P2 + P1$$

Combinação  
→

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |



# Código de Hamming – Exemplo 1



- Maneira correta!
  - Reescrever cada **bit de dado** como a soma das potência de 2

$$M3 = P2 + P1$$

$$M5 = P4 + P1$$

$$M6 = P4 + P2$$

$$M7 = P4 + P2 + P1$$

$$M9 = P8 + P1$$

$$M10 = P8 + P2$$

$$M11 = P8 + P2 + P1$$

Combinação  
→

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

$$P4 = ?$$

$$P8 = ?$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Maneira correta!

- Reescrever cada **bit de dado** como a soma das potência de 2

$$M3 = P2 + P1$$

$$M5 = P4 + P1$$

$$M6 = P4 + P2$$

$$M7 = P4 + P2 + P1$$

$$M9 = P8 + P1$$

$$M10 = P8 + P2$$

$$M11 = P8 + P2 + P1$$

Combinação  
→

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

$$P4 = M5 + M6 + M7$$

$$P8 = M9 + M10 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Outra maneira correta!

$$P1 = M3 + M5 + M7 + M9 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

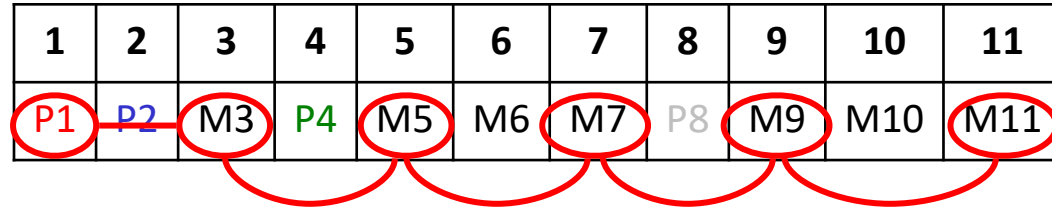
# Código de Hamming – Exemplo 1



- Outra maneira correta!

$$P1 = M3 + M5 + M7 + M9 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |



# Código de Hamming – Exemplo 1



- Outra maneira correta!

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Outra maneira correta!

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Outra maneira correta!

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

$$P4 = M5 + M6 + M7$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Outra maneira correta!

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

$$P4 = M5 + M6 + M7$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |



# Código de Hamming – Exemplo 1



- Outra maneira correta!

$$P1 = M3 + M5 + M7 + M9 + M11$$

$$P2 = M3 + M6 + M7 + M10 + M11$$

$$P4 = M5 + M6 + M7$$

$$P8 = M9 + M10 + M11$$

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  | 11  |
|----|----|----|----|----|----|----|----|----|-----|-----|
| P1 | P2 | M3 | P4 | M5 | M6 | M7 | P8 | M9 | M10 | M11 |

# Código de Hamming – Exemplo 1



- Transmissor quer enviar: 1 1 0 1 0 0 1
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
|   |   |   |   |   |   |   |   |   |    |    |

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
|   |   | 1 |   | 1 | 0 | 1 |   | 0 | 0  | 1  |

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| ? | ? | 1 | ? | 1 | 0 | 1 | ? | 0 | 0  | 1  |

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| ? | ? | 1 | ? | 1 | 0 | 1 | ? | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1$$

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | ? | 1 | ? | 1 | 0 | 1 | ? | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | ? | 1 | ? | 1 | 0 | 1 | ? | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

$$P2 = M3 + M6 + M7 + M10 + M11 \longrightarrow ? = 1 + 0 + 1 + 0 + 1$$

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | 1 | 1 | ? | 1 | 0 | 1 | ? | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

$$P2 = M3 + M6 + M7 + M10 + M11 \longrightarrow ? = 1 + 0 + 1 + 0 + 1 \quad \text{logo } P2 \text{ é } 1$$



# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | 1 | 1 | ? | 1 | 0 | 1 | ? | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

$$P2 = M3 + M6 + M7 + M10 + M11 \longrightarrow ? = 1 + 0 + 1 + 0 + 1 \quad \text{logo } P2 \text{ é } 1$$

$$P4 = M5 + M6 + M7 \longrightarrow ? = 1 + 0 + 1$$

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | ? | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

$$P2 = M3 + M6 + M7 + M10 + M11 \longrightarrow ? = 1 + 0 + 1 + 0 + 1 \quad \text{logo } P2 \text{ é } 1$$

$$P4 = M5 + M6 + M7 \longrightarrow ? = 1 + 0 + 1 \quad \text{logo } P4 \text{ é } 0$$

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | ? | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

$$P2 = M3 + M6 + M7 + M10 + M11 \longrightarrow ? = 1 + 0 + 1 + 0 + 1 \quad \text{logo } P2 \text{ é } 1$$

$$P4 = M5 + M6 + M7 \longrightarrow ? = 1 + 0 + 1 \quad \text{logo } P4 \text{ é } 0$$

$$P8 = M9 + M10 + M11 \longrightarrow ? = 0 + 0 + 1$$

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

$$P2 = M3 + M6 + M7 + M10 + M11 \longrightarrow ? = 1 + 0 + 1 + 0 + 1 \quad \text{logo } P2 \text{ é } 1$$

$$P4 = M5 + M6 + M7 \longrightarrow ? = 1 + 0 + 1 \quad \text{logo } P4 \text{ é } 0$$

$$P8 = M9 + M10 + M11 \longrightarrow ? = 0 + 0 + 1 \quad \text{logo } P8 \text{ é } 1$$

# Código de Hamming – Exemplo 1



- Transmissor quer enviar:
  - Como é pelo código de hamming?

Transmissor envia →

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0  | 1  |

$$P1 = M3 + M5 + M7 + M9 + M11 \longrightarrow ? = 1 + 1 + 1 + 0 + 1 \quad \text{logo } P1 \text{ é } 0$$

$$P2 = M3 + M6 + M7 + M10 + M11 \longrightarrow ? = 1 + 0 + 1 + 0 + 1 \quad \text{logo } P2 \text{ é } 1$$

$$P4 = M5 + M6 + M7 \longrightarrow ? = 1 + 0 + 1 \quad \text{logo } P4 \text{ é } 0$$

$$P8 = M9 + M10 + M11 \longrightarrow ? = 0 + 0 + 1 \quad \text{logo } P8 \text{ é } 1$$



- No receptor:
  - Mensagem original: 01101011001
  - Mensagem recebida: 0110**0**011001 (erro no quinto bit)
  - Como detectar e corrigir o erro?



- No receptor:
  - Mensagem original: 01101011001
  - Mensagem recebida: 0110**0**011001 (erro no quinto bit)
  - Como detectar e corrigir o erro?
- Calcula-se novamente a paridade dos bits verificadores, considerando seu próprio valor:

# Código de Hamming – Exemplo 1



- No receptor:
  - Mensagem original: 01101011001
  - Mensagem recebida: 0110**0**011001 (erro no quinto bit)
  - Como detectar e corrigir o erro?
- Calcula-se novamente a paridade dos bits verificadores, considerando seu próprio valor:

$$P1 + M3 + M5 + M7 + M9 + M11 \longrightarrow 0 + 1 + \textcolor{red}{0} + 1 + 0 + 1 \quad \text{logo } \textcolor{red}{1}$$

$$P2 + M3 + M6 + M7 + M10 + M11 \longrightarrow 1 + 1 + 0 + 1 + 0 + 1 \quad \text{logo } 0$$

$$P4 + M5 + M6 + M7 \longrightarrow 0 + \textcolor{red}{0} + 0 + 1 \quad \text{logo } \textcolor{red}{1}$$

$$P8 + M9 + M10 + M11 \longrightarrow 1 + 0 + 0 + 1 \quad \text{logo } 0$$



# Código de Hamming – Exemplo 1



- No receptor:
  - Mensagem original: 01101011001
  - Mensagem recebida: 0110**0**011001 (erro no quinto bit)
  - Como detectar e corrigir o erro?
- Calcula-se novamente a paridade dos bits verificadores, considerando seu próprio valor:

|                                           |                   |                                          |                           |
|-------------------------------------------|-------------------|------------------------------------------|---------------------------|
| $P_1 + M_3 + M_5 + M_7 + M_9 + M_{11}$    | $\longrightarrow$ | $0 + 1 + \textcolor{red}{0} + 1 + 0 + 1$ | logo $\textcolor{red}{1}$ |
| $P_2 + M_3 + M_6 + M_7 + M_{10} + M_{11}$ | $\longrightarrow$ | $1 + 1 + 0 + 1 + 0 + 1$                  | logo 0                    |
| $P_4 + M_5 + M_6 + M_7$                   | $\longrightarrow$ | $0 + \textcolor{red}{0} + 0 + 1$         | logo $\textcolor{red}{1}$ |
| $P_8 + M_9 + M_{10} + M_{11}$             | $\longrightarrow$ | $1 + 0 + 0 + 1$                          | logo 0                    |

Qual o bit com erro?

# Código de Hamming – Exemplo 1



- No receptor:
  - Mensagem original: 01101011001
  - Mensagem recebida: 0110**0**011001 (erro no quinto bit)
  - Como detectar e corrigir o erro?
- Calcula-se novamente a paridade dos bits verificadores, considerando seu próprio valor:

|                                 |                   |                                          |                           |
|---------------------------------|-------------------|------------------------------------------|---------------------------|
| $P1 + M3 + M5 + M7 + M9 + M11$  | $\longrightarrow$ | $0 + 1 + \textcolor{red}{0} + 1 + 0 + 1$ | logo $\textcolor{red}{1}$ |
| $P2 + M3 + M6 + M7 + M10 + M11$ | $\longrightarrow$ | $1 + 1 + 0 + 1 + 0 + 1$                  | logo 0                    |
| $P4 + M5 + M6 + M7$             | $\longrightarrow$ | $0 + \textcolor{red}{0} + 0 + 1$         | logo $\textcolor{red}{1}$ |
| $P8 + M9 + M10 + M11$           | $\longrightarrow$ | $1 + 0 + 0 + 1$                          | logo 0                    |

Qual o bit com erro?

- $0101_2 \Rightarrow 5_{10}$

# Código de Hamming – Exemplo 1



- No receptor:
  - Mensagem original: 01101011001
  - Mensagem recebida: 0110**0**011001 (erro no quinto bit)
  - Como detectar e corrigir o erro?
- Calcula-se novamente a paridade dos bits verificadores, considerando seu próprio valor:

|                                 |                   |                         |               |
|---------------------------------|-------------------|-------------------------|---------------|
| $P1 + M3 + M5 + M7 + M9 + M11$  | $\longrightarrow$ | $0 + 1 + 0 + 1 + 0 + 1$ | logo <b>1</b> |
| $P2 + M3 + M6 + M7 + M10 + M11$ | $\longrightarrow$ | $1 + 1 + 0 + 1 + 0 + 1$ | logo 0        |
| $P4 + M5 + M6 + M7$             | $\longrightarrow$ | $0 + 0 + 0 + 1$         | logo <b>1</b> |
| $P8 + M9 + M10 + M11$           | $\longrightarrow$ | $1 + 0 + 0 + 1$         | logo 0        |

E se o erro  
for em P1?

Qual o bit com erro?

- $0101_2 \Rightarrow 5_{10}$

# Código de Hamming – Exemplo 2?



- Transmissor quer enviar: 1001101
  - Codifique a mensagem pelo código de hamming (11, 7)?
  - Simule a recepção da mensagem sem erros e com erro no bit 7

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| ? | ? | 1 | ? | 0 | 0 | 1 | ? | 1 | 1  | 0  |

# Código de Hamming – Exemplo 2?



- Transmissor quer enviar: 1001101
  - Codifique a mensagem pelo código de hamming (11, 7)?
  - Simule a recepção da mensagem sem erros e com erro no bit 7

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|----|----|
| ? | ? | 1 | ? | 0 | 0 | 1 | ? | 1 | 1  | 0  |

**P1** =  $M_3 + M_5 + M_7 + M_9 + M_{11} \Rightarrow 1 + 0 + 1 + 1 + 0$ , logo **P1 1**

**P2** =  $M_3 + M_6 + M_7 + M_{10} + M_{11} \Rightarrow 1 + 0 + 1 + 1 + 0$ , logo **P2 1**

**P4** =  $M_5 + M_6 + M_7 \Rightarrow 0 + 0 + 1$ , logo **P4 1**

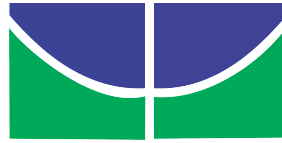
**P8** =  $M_9 + M_{10} + M_{11} \Rightarrow 1 + 1 + 0$ , logo **P8 0**

**11110010110**

**Dúvidas?**



- BRANCO, K. R. L. C., MOREIRA, P. H. G. BRANCO, L. H. C., BARBOSA, E. F. e MEIRELES, M. T. **Redes de computadores** - Da teoria à prática com NETKIT. 1ª ed. Elsevier, 2014.
- COMER, Douglas. **Redes de computadores e internet**: abrange transmissão de dados, ligações inter-redes, web e aplicações. 4ª ed. Porto Alegre: Bookman, 2007.
- KUROSE, J. F.; ROSS, K. W. **Redes de Computadores e a Internet**: Uma Abordagem Top-down. 5ª ed. São Paulo: Prentice Hall, 2010.
- TANENBAUM, A. S. **Redes de computadores**. 5ª ed. Rio de Janeiro: Elsevier, 2011.
- TORRES, Gabriel. **Redes de computadores Curso completo**. 2ª ed. Rio de Janeiro: Novaterra Editora, 2016.



Teleinformática e Redes 1

# **Camada de Enlace de Dados: Detecção e Correção de Erros**

Prof.º Geraldo P. Rocha Filho

geraldof@unb.br

Brasília  
2019